Voice-Driven Robotic Manipulation System (Phase 1)

- Team 1, Advanced Mechatronics (Aditya, Sparsh & Ritika)

Abstract

This project presents the design and experimental validation of a voice-driven robotic manipulation system integrating speech interpretation, calibrated vision, kinematic modelling, and embedded servo control. The objective of Phase 1 is to develop a fixed robotic arm capable of executing pick-and-place tasks based on spoken commands and vision-based object localization.

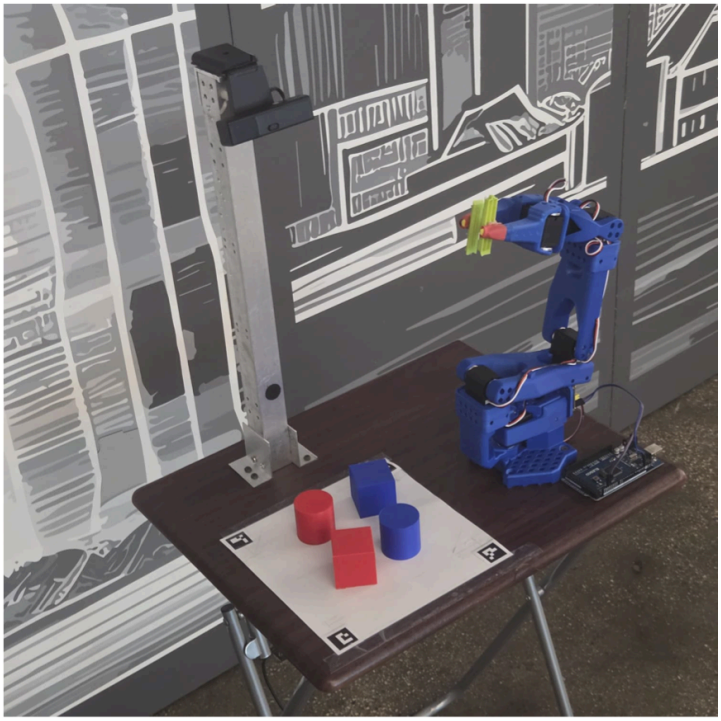
A calibrated monocular camera (IC840) is rigidly mounted above the workspace to detect and localize objects of known geometry. Intrinsic parameters and distortion coefficients are estimated using standard camera calibration techniques, and performance is validated through reprojection error analysis. Object detection is performed using colour segmentation and contour-based shape classification in OpenCV. Extracted image coordinates are mapped to real-world planar coordinates, enabling physically meaningful motion planning.

Inverse kinematics is implemented to convert target object positions into joint-space commands for servo actuators. The complete processing pipeline runs on a laptop and integrates speech-to-text processing, perception, and control.

Experimental results demonstrate successful object detection, coordinate estimation, and voice-triggered pick-and-place execution. The system establishes a structured framework for future integration with LLM-based task planning and mobile robotic navigation.

1. Introduction

Human-robot interaction in manipulation tasks can be simplified using natural language commands. This project implements a practical end-to-end prototype where a user speaks an object command (for example, “red cube”), and the robot identifies the object, computes a corresponding grasp target, and executes a pick-and-place routine.



Phase 1 focuses on establishing a stable perception-to-action pipeline for a fixed-base arm. The work combines:

- Speech command interpretation,
- Vision-based object localization in a calibrated workspace,
- Mapping from workspace coordinates to arm command packets,
- Serial communication with a low-level servo controller.

2. Problem Statement

The project addresses the challenge of reliable voice-driven manipulation in a constrained workspace using low-cost sensing and embedded actuation. The core problem is to transform a spoken object command into a physically correct pick-and-place action through a robust perception-to-control pipeline.

The specific requirements are: 1. Build a ROS 2-based modular architecture connecting perception, speech, planning, and control. 2. Detect and classify target objects by color and shape using monocular vision. 3. Map detections to workspace coordinates and select grasp targets. 4. Convert targets into valid joint step commands for hardware execution. 5. Validate voice-triggered pick-and-place behavior through experiments.

3. Solution Approach

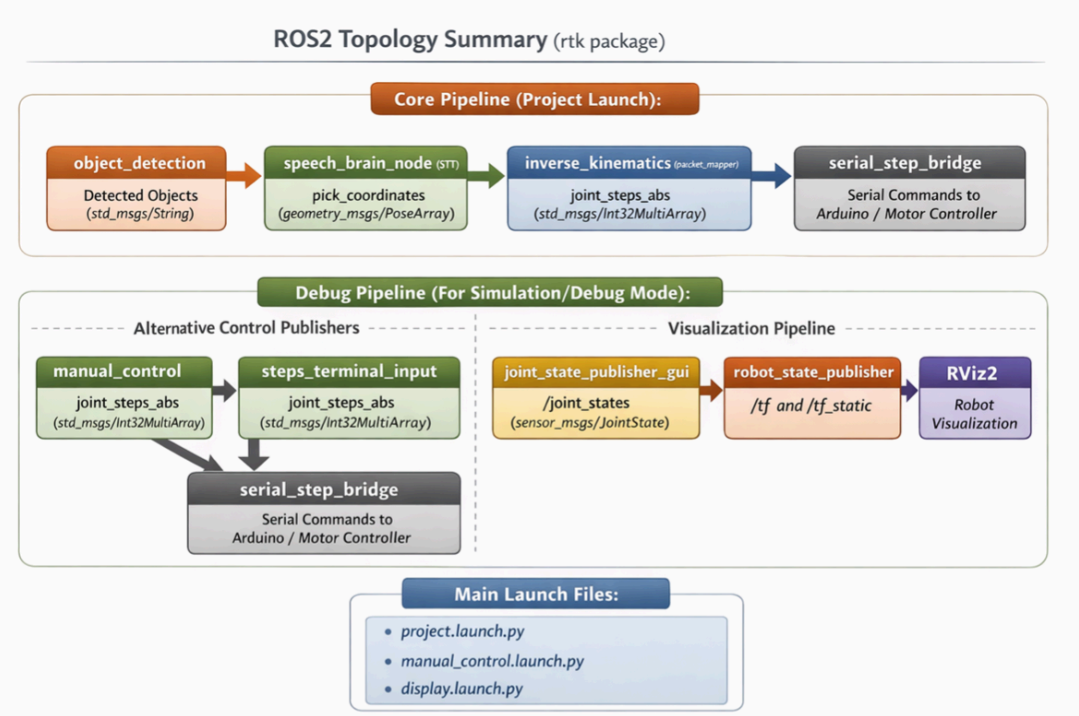
3.1 Hardware Design

- Host computer (Linux laptop) running ROS 2 nodes.
- Fixed robotic arm with servo actuation.
- Monocular USB camera (IC840) mounted above the workspace.
- Embedded controller (Arduino-compatible) connected over serial.

3.2 Software Design

- ROS 2 for node orchestration and inter-process messaging.
- Python for rapid development of processing nodes.
- OpenCV + NumPy for calibration and object detection.
- SpeechRecognition + Google STT for command transcription.
- gTTS + playsound for spoken feedback.
- pyserial for serial command forwarding.

3.3 ROS 2 Node Topology



Core execution pipeline: 1. `object_detection` publishes `detected_objects` (`std_msgs/String`). 2. `speech_brain_node` (`stt`) subscribes to detected labels and publishes `pick_coordinates` (`geometry_msgs/PoseArray`). 3. `inverse_kinematics` (`packet_mapper`) subscribes to pick coordinates and publishes `joint_steps_abs` (`std_msgs/Int32MultiArray`). 4. `serial_step_bridge` subscribes to joint steps and transmits serial commands to the controller.

Additional control modes: - `manual_control` and `steps_terminal_input` can publish `joint_steps_abs` directly for testing. Visualization support: - `joint_state_publisher_gui`, `robot_state_publisher`, and `rviz2` support model/pose visualization.

3.4 Processing Pipeline (Detailed methods can be moved to Appendix)

3.4.1 Camera Calibration and Workspace Mapping

The camera is calibrated to estimate intrinsic matrix and lens distortion coefficients. Calibrated parameters are used to undistort incoming frames. Four ArUco markers define workspace corners, enabling homography estimation between image space and a planar workspace coordinate frame.

This homography allows conversion of object detections from pixel coordinates to metric-like workspace coordinates (centimeters), which are used downstream for motion selection.

3.4.2 Object Detection and Classification

Object perception is implemented with OpenCV and follows a deterministic pipeline: 1. RGB frame capture and optional undistortion. 2. HSV color thresholding for red and blue masks. 3. Morphological filtering (open/close) to suppress noise. 4. Edge extraction and contour detection. 5. Shape classification using contour approximation, circularity, and aspect ratio.

Detected objects are represented as labels (for example, `red cube`) with workspace coordinates, published as a compact ROS topic payload.

3.4.3 Speech Interface and Command Resolution

The speech node continuously listens for user commands, transcribes audio, and extracts a valid object phrase from predefined color and shape keywords. It cross-references the spoken target with currently detected objects. If found, the node publishes a `PoseArray` target for execution and provides voice feedback to the user.

3.4.4 Inverse Kinematics and Motion Packet Generation

The inverse kinematics node maps target workspace coordinates to the nearest grid location and corresponding joint command packet. To improve spatial resolution, the command grid is densified by interpolation between base calibration points. The node then executes a staged pick-and-place sequence:

1. Move arm to selected object approach/grasp pose.
2. Close gripper.
3. Return through intermediate safe poses.
4. Move to drop pose.
5. Open gripper and reset. Commands are published as absolute joint step arrays.

3.4.5 Serial Bridge to Embedded Controller

The serial bridge validates incoming six-element step arrays and forwards them in controller-compatible text format (`J:x,x,x,x,x,x`) over USB serial. This cleanly decouples ROS-level planning logic from low-level motor execution.

4. Tests and Results

4.1 Test Setup

- Workspace instrumented with ArUco corner markers.
- Test objects: red/blue cubes and cylinders.
- Camera fixed above the workspace with calibrated intrinsics.
- ROS 2 launch used to run integrated pipeline (`project.launch.py`).
- Manual/Debug mode (`manual_control.launch.py`) used for actuator and communication verification.

4.2 Results

4.2.1 Functional Validation

The integrated system successfully performs: - Real-time object detection and label publication, - Voice command understanding for supported color-shape combinations, - Coordinate-to-command conversion, - End-to-end pick-and-place actuation triggered by speech.

4.2.2 Observed Performance

- Calibration-enabled mapping provides stable coordinate estimation within the defined workspace.
- Topic-based ROS modularity simplifies debugging and subsystem isolation.
- The serial bridge reliably transmits commands and enforces message sanity checks.

4.3 Test-Observed Constraints

- Speech understanding is constrained to a fixed vocabulary.
- Perception is sensitive to lighting variation and occlusion.
- Grasp planning is discrete/grid-based rather than continuous IK optimization.
- Motion sequencing uses timed delays and does not yet include force/contact feedback.

5. Discussion

The system demonstrates that a modular ROS 2 architecture can support practical voice-driven manipulation with monocular perception and low-level serial control. The strongest outcomes are subsystem decoupling, testability, and successful end-to-end command execution for supported object classes.

Key trade-offs observed during experiments: - Robustness versus simplicity: deterministic color/shape heuristics are fast and transparent but sensitive to lighting and occlusions. - Discrete control versus precision: grid-mapped/interpolated packets simplify execution but limit continuous motion optimality. - Open-loop sequencing versus reliability: timer-based motion stages are easy to deploy but would benefit from closed-loop confirmation.

6. Conclusions

Phase 1 demonstrates a complete voice-to-action robotic manipulation pipeline integrating speech, vision, coordinate mapping, and embedded control. The system achieves consistent command-triggered pick-and-place behavior and provides a practical baseline for higher-level autonomy.

6.1 Future Work

Planned next steps include: - LLM-based natural language task planning beyond fixed command templates, - Continuous kinematic solving with trajectory smoothing, - Robust multi-object tracking and confidence-aware selection, - Closed-loop grasp verification and failure recovery, - Mobile platform integration for navigation + manipulation workflows.

7. References

1. OpenCV Documentation, “Image Processing (Color Spaces, Morphology, Contours),” https://docs.opencv.org/
2. ROS 2 Documentation, “Topics, Nodes, and rclpy API,” https://docs.ros.org/
3. Garrido-Jurado, S. et al., “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, 2014.
4. Corke, P., *Robotics, Vision and Control*, 2nd ed., Springer, 2017.
5. Hartley, R. and Zisserman, A., *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
6. Siciliano, B. et al., *Robotics: Modelling, Planning and Control*, Springer, 2009.

Appendix A: Reproducibility Notes

Here's the github repo for our project: GitHub Relevant launch files and modules are available in the package: - `launch/project.launch.py` - `launch/manual_control.launch.py` - `launch/object_detection.launch.py` - `rtk/object_detection.py` - `rtk/stt.py` - `rtk/inverse_kinematics.py` - `rtk/serial_step_bridge.py`

This structure supports modular testing of each subsystem and integrated end-to-end execution.